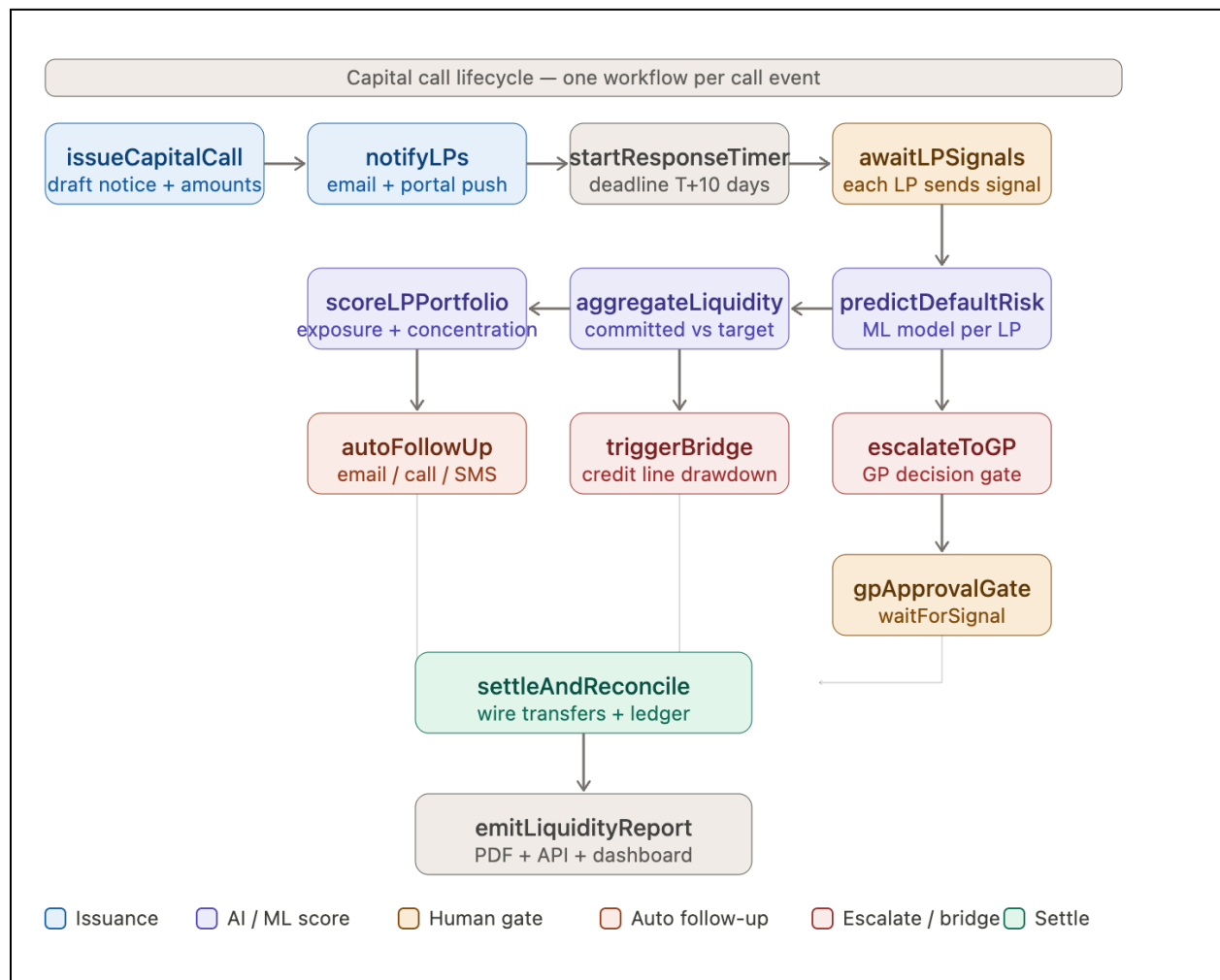


Use-Case — Asset Management: Intelligent Capital Call & Liquidity Orchestration

Business idea: A GP (General Partner) issues a capital call. The workflow notifies all LPs, waits for their commitment signals, runs ML risk scoring on each LP (Limited Partner), autonomously follows up overdue LPs, triggers a credit line drawdown if the liquidity gap is too large, escalates high-risk LPs to the GP for a manual decision, then settles and emits a regulatory-grade report.



Temporal components

Workflow: `CapitalCallWorkflow` (parent) + `LPResponseWorkflow` (child, one per LP)

Parent input: `{callId, fundId, targetAmountUSD, lpList: [{lpId, commitmentUSD, email}], deadlineDays: 10}`

Parent sequence:

1. `issueCapitalCall` — creates call record, computes each LP's draw amount
2. `notifyLPs` — fan-out batch activity sends emails via SES stub
3. Starts one child `LPResponseWorkflow` per LP concurrently
4. `Promise.all` with a `deadline` timer — collects all LP signals
5. `aggregateLiquidity` — computes committed vs target gap
6. `predictDefaultRisk` — ML model scores each LP's default probability
7. Branch on gap: if gap > 10% → `triggerBridge` (credit line drawdown)
8. For LPs with risk score > 0.7: `escalateToGP` → `gpApprovalGate` (human signal)
9. After all branches resolve: `settleAndReconcile` — wire transfer stubs
10. `emitLiquidityReport` — PDF + JSON report

Child workflow `LPResponseWorkflow`:

1. `waitForSignal("lpCommitment")` — waits for LP portal webhook
2. On timeout: triggers `autoFollowUp` escalation sequence (email D+3, call D+7, legal notice D+10)
3. Returns `{lpId, status: "committed" | "defaulted" | "deferred", amountUSD}`

Activities:

Activity	What it does	Mock
<code>issueCapitalCall</code>	Creates call record in fund admin system	Inserts into Postgres
<code>notifyLPs</code>	Batch SES email with secure portal links	Logs per-LP payload
<code>autoFollowUp</code>	Sends tiered escalation communications	Logs with delay simulation
<code>predictDefaultRisk</code>	Scikit-learn model scores LP creditworthiness	Returns synthetic score from LP attributes
<code>aggregateLiquidity</code>	Sums committed amounts, computes gap	Pure computation activity
<code>scoreLPPortfolio</code>	Concentration risk + correlation analysis	Returns portfolio risk metrics

<code>triggerBridge</code>	POSTs drawdown request to mock credit facility	Returns confirmation + fee
<code>escalateToGP</code>	Slack alert with LP summary to GP channel	Logs escalation
<code>settleAndReconcile</code>	Wire transfer stubs + double-entry ledger update	Inserts into mock ledger table
<code>emitLiquidityReport</code>	Generates PDF + JSON report	Writes to <code>/reports/{callId}.pdf</code>

Signal endpoints:

None

`POST /api/capital-calls/{callId}/lp-response`

Body: { "lpId": "lp-42", "amount": 5000000, "runId": "..." }

`POST /api/capital-calls/{callId}/gp-decision`

Body: { "lpId": "lp-42", "action": "waive" | "enforce", "gpName": "..." }

Demo driver: Pre-seeds 10 mock LPs. 7 respond immediately, 2 are "deferred" (triggering `autoFollowUp`), and 1 is "high risk" (triggering `escalateToGP`). LP responses fire on a staggered timer so the whole flow is visible in real time.

Docker compose services: `temporal-server`, `temporal-ui`, `postgres` (fund ledger), `worker`, `api-server`, `mock-ses` (email log viewer), `mock-credit-facility` (HTTP stub), `ml-scorer` (Flask app with scikit-learn model), `report-generator` (WeasyPrint PDF service), `demo-driver`.